

Taller 2

Asignatura: Minería de Datos

Código: 2016325

Porcentaje: 25 %

Fecha limite de entrega: 4 de junio de 2026.

Modalidad: individual.

Horario de atención: Viernes 10:00-12:00.

Temas: Clases 1-11.

1. Objetivo

Construir una aplicación en Shiny o Streamlit que permita consultar, visualizar y actualizar la base de datos SQLite creada en el Taller 1.

2. Actividad

En el taller 1 realizamos un proceso de web scraping sobre revistas Q1 e identificamos cuáles estaban relacionadas con Machine Learning, IA Generativa y Estadística; posteriormente, se almacenó la información en una base de datos SQLite. En este segundo taller deberán desarrollar una aplicación interactiva utilizando Shiny (R) o Streamlit (Python) para explorar, visualizar y actualizar dicha base de datos.

La aplicación deberá funcionar como un dashboard analítico orientado al proceso KDD visto en clase, integrando adquisición, almacenamiento, consulta y visualización de datos.

2.1. Requerimientos de la Aplicación

La aplicación desarrollada en **Shiny** o **Streamlit** deberá incluir como mínimo los siguientes componentes:

2.1.1. Conexión a SQLite

- Conectarse a la base de datos SQLite creada en el Taller 1.

- Leer una o más tablas de la base de datos.
- Permitir consultas dinámicas sobre los datos almacenados.

2.1.2. Sidebar interactivo

La interfaz deberá incluir un panel lateral (*sidebar*) con filtros para:

- rango de fechas;
- tema o categoría;
- autor;
- DOI;
- búsqueda por palabras clave o título.

2.1.3. Widgets interactivos

La aplicación deberá contener al menos tres widgets interactivos, por ejemplo:

- selector de fechas;
- listas desplegables;
- campos de texto;
- botones de acción;
- tablas interactivas.

2.1.4. Indicadores descriptivos

El dashboard deberá mostrar al menos 5 indicadores calculados a partir de la base de datos, tales como:

- número total de artículos;
- promedio de autores por artículo;
- promedio de citas;
- promedio de referencias;

- número de artículos por temática;
- artículo más citado;
- artículo más descargado.

2.1.5. Visualizaciones interactivas

La aplicación deberá incluir al menos dos gráficos interactivos utilizando:

- `highcharter` en Shiny; o
- `plotly` en Streamlit.

Algunas visualizaciones sugeridas son:

- evolución temporal de publicaciones;
- artículos por categoría;
- top autores;
- distribución de citas;
- descargas por temática.

2.1.6. Tabla interactiva

Mostrar una tabla dinámica con los artículos filtrados, incluyendo información como:

- título;
- autores;
- fecha;
- tema;
- DOI;
- citas;
- descargas.

2.1.7. Actualización automática mediante scraping

La aplicación deberá incluir un botón para buscar artículos nuevos mediante scraping:

- si existen nuevos artículos (dado que en el Taller 1 se trabajó con artículos de 2025, pueden existir artículos de 2026), estos deberán mostrarse en pantalla y almacenarse automáticamente en SQLite;
- la aplicación deberá informar al usuario el número de artículos nuevos encontrados;
- en caso de no existir nuevos artículos, la aplicación deberá reconsultar los últimos cinco artículos almacenados mediante scraping para verificar actualizaciones.

2.2. Entrega

Debe subir al **GitHub del curso**, mediante un **Pull Request**, (en la carpeta individual) lo siguiente:

- archivo `app.R` o archivos `ui.R` y `server.R` en el caso de Shiny;
- archivo `app.py` en el caso de Streamlit;
- opcionalmente, archivos auxiliares necesarios para el estilo y funcionamiento del app.

3. Despliegue

3.1. Publicación de la aplicación en `shinyapps.io`

Para publicar la aplicación desarrollada en Shiny se debe seguir el siguiente procedimiento:

1. Crear una cuenta en <https://www.shinyapps.io/>.
2. Iniciar sesión en la plataforma y acceder al panel principal.
3. Crear un nuevo *token* de acceso:
 - Ir a **Account >Tokens**.
 - Seleccionar **Show Secret**.
 - Copiar el código mostrado.
4. Instalar en R las librerías necesarias:

```
install.packages("shiny")
install.packages("rsconnect")
```

5. Abrir en RStudio el proyecto o carpeta donde se encuentra la aplicación Shiny.
6. Configurar la conexión con `shinyapps.io` utilizando el token generado:

```
library(rsconnect)

rsconnect::setAccountInfo(
  name='USUARIO',
  token='TOKEN',
  secret='SECRET'
)
```

7. Verificar que la aplicación funcione localmente ejecutando:

```
shiny::runApp()
```

8. Publicar la aplicación ejecutando:

```
rsconnect::deployApp()
```

9. Esperar a que finalice el proceso de despliegue.
10. Copiar el enlace generado por `shinyapps.io` y verificar que la aplicación funcione correctamente desde el navegador.
11. Incluir el enlace de la aplicación desplegada en el `README.md` o en la entrega del taller.

3.2. Publicación de una aplicación en Streamlit Cloud

Streamlit Community Cloud permite publicar aplicaciones de manera gratuita utilizando GitHub.

3.2.1. Requisitos previos

- Tener una cuenta en GitHub.
- Tener una cuenta en <https://streamlit.io/cloud>.
- Tener el proyecto subido a un repositorio de GitHub.

3.2.2. Paso a paso

1. Crear un repositorio en GitHub y subir los archivos de la aplicación:

- `app.py`
- base de datos SQLite
- `requirements.txt`
- archivos auxiliares necesarios

2. Crear el archivo `requirements.txt` con las librerías utilizadas. Por ejemplo:

```
streamlit
pandas
plotly
sqlalchemy
beautifulsoup4
requests
```

3. Ingresar a:

<https://streamlit.io/cloud>

4. Iniciar sesión utilizando la cuenta de GitHub.
5. Seleccionar la opción `New App`.
6. Elegir:
 - el repositorio de GitHub;
 - la rama (`main` o `master`);
 - el archivo principal de la aplicación (`app.py`).
7. Hacer clic en `Deploy`.
8. Esperar a que finalice el proceso de despliegue.
9. Verificar que la aplicación funcione correctamente desde el enlace generado.
10. Incluir el enlace de la aplicación publicada en la entrega del taller.

3.2.3. Plan gratuito

La plataforma ofrece un plan gratuito que permite:

- publicar aplicaciones públicas;
- conectarse directamente con GitHub;
- actualizar automáticamente la aplicación cuando se hagan cambios en el repositorio;
- compartir la aplicación mediante un enlace web.

3.2.4. Recomendaciones

- No subir archivos demasiado grandes al repositorio.
- Verificar que el archivo `requirements.txt` incluya todas las dependencias.
- Probar la aplicación localmente antes de desplegarla:

```
streamlit run app.py
```

4. Peso para cada punto

Criterio	Porcentaje
Scraping de nuevos papers	50 %
Despliegue	10 %
Widgets y Gráficas	15 %
Indicadores y estilo	25 %